

CSCI 650 - Algorithms and Computability

Assignment 3

Sean Angrisani

September 30, 2026

Solutions to the written questions on this assignment should be submitted via PDF to Canvas before **October 4th at 11:59 pm**. Your Turing machine simulator and your implementation of the CYK algorithm should be submitted to INGIous by the same deadline. Make sure to justify your answers.

I highly encourage you to collaborate with one another. However, you must write up your own solutions **independently**. Feel free to communicate via [Discord](#) and to post questions on the appropriate forum in Canvas. Do not post solutions.

Have fun!

Problems

1. In this question we will write a simple function to simulate a Turing machine and use it to investigate $BB(2)$, the second busy beaver number.
 - (a) (10 pts) Write a function `simulateTM(delta, A, w)` which takes a transition function, a set of accepting states, and a string as input. This function should return a string associated with the final tape for the Turing machine along with a boolean indicating whether or not `w` is accepted. We will assume that `A` is the start state and that `b` is the blank symbol. Submit your code on INGIous and include a code snippet of this function in your PDF submission. `turingMachineTests.py`, a script meant to help with local testing, is available on Canvas.
 - (b) (15 pts) Consider the [busy beaver problem](#) as discussed in class. Write code to consider all Turing machines with two states, a binary input alphabet, and a tape initially filled with 0s. Augment your code from part (a) to count the number of steps each machine takes before halting up to a maximum of 100 steps. Create a table similar to the one below indicating the number and fraction of machines that take i steps to finish. Include a snippet of the code you used to find these values.

Steps	0	1	2	3	4	5	6	≥ 100
Number	3	8	4	12	42	112	57	239
Fraction	0.1	0.04	0.2	0.07	0.06	0.03	0.05	0.45

- (c) ($BB(n)$ pts for $n \geq 6$) Extra credit - Find a Turing machine associated with $BB(n)$ and prove that it is correct.
2. (10 pts) Context-free grammars play an important role in many areas of computer science including in programming languages and compilers, natural language processing, and parsing among others. Write a function `cyk(G, w)` that takes a context-free grammar and a string as input and returns a boolean indicating whether or not the string is in the language associated with the context-free

grammar along with a table filled in according to the CYK algorithm as discussed in class. Submit your code on INGINious and include a code snippet of this function in your PDF submission. `cykTests.py`, a script meant to help with local testing, is available on Canvas.

3. Recall the definitions of the vertex cover and independent set problems.

Definition (Vertex Cover). *Given an undirected graph $G = (V, E)$, a **vertex cover** of G is a subset of vertices $C \subseteq V$ such that, for every edge $\{u, v\} \in E$, either $u \in C$ or $v \in C$.*

The Vertex Cover Problem

Input: An undirected graph $G = (V, E)$ and an integer k .

Output: True if and only if G has a vertex cover of size k or less.

Definition (Independent Set). *Given an undirected graph $G = (V, E)$, an **independent set** is a subset of vertices $I \subseteq V$ such that no two elements of I are adjacent. In particular, if $u, v \in I$, then $\{u, v\} \notin E$.*

The Independent Set Problem

Input: An undirected graph $G = (V, E)$ and an integer k .

Output: True if and only if G contains an independent set of size k or more.

- (a) (10 pts) Write pseudocode (using Python syntax) for a function `checkVertexCover(G, k, C)` that returns **True** if C is a vertex cover of G of size k or less and **False** otherwise. Analyze the asymptotic runtime of this function.
- (b) (10 pts) Write pseudocode (using Python syntax) for a function `findIndependentSet(G)` that finds an independent set of maximum size in G . You may assume access to a function `findVertexCover(G)` that is guaranteed to return a vertex cover of G of minimum size in polynomial time. In particular, `findIndependentSet(G)` must convert an arbitrary independent set instance to an instance of vertex cover. Analyze the asymptotic runtime of this function.

4. The set cover problem is defined as follows.

Definition (Set Cover). *Given a set of elements U (often called the universe) and subsets $S_1, S_2, \dots, S_m \subseteq U$, $C \subseteq \{1, 2, \dots, m\}$ is a **set cover** of U if $\bigcup_{i \in C} S_i = U$.*

The Set Cover Problem

Input: A universe U , sets $S_1, S_2, \dots, S_m \subseteq U$, and an integer k .

Output: True if and only if there is a cover C of U such that $|C| \leq k$.

- (a) (10 pts) Write pseudocode (using Python syntax) for a function `checkSetCover(U, S, k, C)` that returns **True** if C is a set cover of U of size k or less and **False** otherwise. Analyze the asymptotic runtime of this function.
- (b) (10 pts) Write pseudocode (using Python syntax) for a function `findVertexCover(G)` that finds a vertex cover of minimum size in G . You may assume access to a function `findSetCover(U, S)` that is guaranteed to return a minimum set cover of U in polynomial time. In particular, `findVertexCover(G)` must convert an arbitrary vertex cover instance to an instance of set cover. Analyze the asymptotic runtime of this function.

Solutions: Sean Angrisani

1(a): Turing machine simulator

The tape extends in both directions. Unstored cells are blank. Each transition updates one tape cell. The head then moves left or right. Acceptance stops execution immediately. Missing transitions halt the machine without accepting. Return acceptance alongside all the final tape contents. Only outer blanks are removed. Internal blanks stay. An entirely blank tape returns an empty string.

The supplied skeleton determines the return order. Its original fallback remains in place. Nonhalting machines can run forever. No arbitrary cutoff is imposed here.

```
def simulateTM(delta, A, w):
    tape = dict(enumerate(w))
    state = 'A'
    head = 0
    while state not in A:
        symbol = tape.get(head, 'b')
        if (state, symbol) not in delta:
            break
        state, symbol, direction = delta[(state, symbol)]
        tape[head] = symbol
        head += 1 if direction == 'R' else -1
    result = ''
    if tape:
        result = ''.join(tape.get(i, 'b')
                          for i in range(min(tape), max(tape) + 1))
        result = result.strip('b')
    if state in A:
        return True, result
    if result:
        return False, result
    return False, ''
```

1(b): Two-state busy beaver experiment

Use two working states.

$$Q = \{A, B\}, \quad \Gamma = \{0, 1\}, \quad q_{\text{halt}} = H.$$

State A starts each run. Every tape cell initially contains zero. The halt state is additional. Each transition writes, moves, and changes state. Halting transitions also count as steps. Every state-symbol pair has twelve possible transitions. There are four such pairs. Thus, the enumeration contains $12^4 = 20,736$ machines. Mirrors count separately. Unreachable transition choices also remain distinct.

Steps	Number	Fraction
0	0	0/20736
1	6912	6912/20736
2	2304	2304/20736
3	384	384/20736
4	128	128/20736
5	16	16/20736
6	40	40/20736
7–99	0	0/20736
≥ 100 or unresolved	10952	10952/20736
Total	20736	1

The last bucket includes unresolved machine runs. Halting after exactly 100 steps also belongs here. The largest observed halting time is six. A cutoff alone cannot prove nontermination. Maximality remains unproved. The known step maximum agrees: $BB(2) = 6$. See Aaronson's [The Busy Beaver Frontier](#) for context. The next page contains the counting code.

1(c): Optional extra credit

This optional problem remains unattempted.

1(b): Counting code

```
from collections import Counter
from itertools import product

def countSteps(delta, max_steps=100):
    tape = {}
    state = 'A'
    head = 0
    for steps in range(max_steps):
        if state == 'H':
            return steps
        state, symbol, direction = delta[(state, tape.get(head, '0'))]
        tape[head] = symbol
        head += 1 if direction == 'R' else -1
    if state == 'H':
        return max_steps
    return None

def busyBeaverCounts(max_steps=100):
    keys = list(product('AB', '01'))
    choices = list(product('ABH', '01', 'LR'))
    counts = Counter()
    for transitions in product(choices, repeat=4):
        delta = dict(zip(keys, transitions))
        steps = countSteps(delta, max_steps)
        bucket = max_steps if steps is None else steps
        counts[bucket] += 1
    return counts
```

2: CYK algorithm

The supplied generator uses Chomsky normal form. Each production contains one terminal or two variables. The grammar representation follows that generator.

$$G = (V, T, P, S).$$

Here, P maps variables to production sets. The table uses inclusive substring endpoints. Thus, `table[i][j]` describes `w[i:j+1]`. Each cell stores possible generating variables. Diagonal entries match characters. Longer substrings consider every possible split point. Matching rules contribute their head variables. Unused entries stay empty. The return value contains acceptance and table.

```
def cyk(G, w):
    V, T, P, S = G
    n = len(w)
    table = [[set() for _ in range(n)] for _ in range(n)]
    if n == 0:
        return '' in P.get(S, set()), table
    for i in range(n):
        for head, bodies in P.items():
            if w[i] in bodies:
                table[i][i].add(head)
    for length in range(2, n + 1):
        for i in range(n - length + 1):
            j = i + length - 1
            for k in range(i, j):
                for head, bodies in P.items():
                    for body in bodies:
                        if (len(body) == 2
                            and body[0] in table[i][k]
                            and body[1] in table[k + 1][j]):
                            table[i][j].add(head)
    return S in table[0][n - 1], table
```

Single-character matches establish the induction base. Assume shorter entries are correct. Each split combines two correct shorter entries. Every binary derivation splits this way. Therefore, each computed cell contains exactly its generators. Acceptance checks the complete string's cell. Empty input uses the start variable's empty production. Its table contains no cells.

Let n denote the input length. Let p count grammar productions. The expected running time is $O((p + 1)(n^3 + 1))$. This assumes constant-time hash operations. Storage is $O(n^2|V|)$.

3(a): Checking vertex covers

Represent the graph as (V, E) . Vertices and certificates use Python sets. Edges pair vertices.

```
def checkVertexCover(G, k, C):
    V, E = G
    if len(C) > k or not C.issubset(V):
        return False
    for u, v in E:
        if u not in C and v not in C:
            return False
    return True
```

The first check rejects oversized or invalid certificates. Each remaining edge needs one selected endpoint. Any uncovered edge causes rejection. Otherwise, every edge is covered within budget. This matches the vertex cover definition.

Membership checks take expected constant time here. The expected runtime is $O(1 + |C| + |E|)$. Valid certificates satisfy $|C| \leq |V|$. Hence, $O(1 + |V| + |E|)$ bounds valid inputs. Auxiliary space remains constant.

3(b): Finding maximum independent sets

```
def findIndependentSet(G):
    V, E = G
    C = findVertexCover(G)
    return V - C
```

The reduction preserves the original graph. Decision instances transform as follows.

$$(G, k)_{\text{IS}} \mapsto (G, |V| - k)_{\text{VC}}.$$

Let C denote any vertex cover. No edge can join two vertices outside C . Consequently, $V \setminus C$ is independent. Conversely, take any independent set I . Every edge has an endpoint outside I . Thus, $V \setminus I$ covers all edges. Together, their complementary sizes always sum to $|V|$. Minimizing one therefore maximizes the other. The oracle supplies the required minimum cover.

Let T_{VC} denote the oracle runtime. The total runtime is $T_{\text{VC}}(G) + O(1 + |V|)$. This is polynomial under the stated assumption. The returned set needs $O(|V|)$ space.

4(a): Checking set covers

Use sets for the universe and certificate. Subset indices begin at one. Store subsets in a dictionary S . Its keys are $1, \dots, m$.

```
def checkSetCover(U, S, k, C):
    if len(C) > k:
        return False
    covered = set()
    for i in C:
        if i not in S:
            return False
        covered.update(S[i])
    return covered == U
```

The initial check enforces the size limit. Invalid subset indices cause rejection. The union collects every selected subset's elements. Equality checks completeness. Therefore, acceptance exactly matches a valid cover.

Define the selected input size below.

$$L_C = \sum_{i \in C} |S_i|.$$

Expected runtime is $O(1 + |C| + L_C + |U|)$. Hash operations take expected constant time. Auxiliary space is $O(1 + |U|)$. This uses the given assumption $S_i \subseteq U$.

4(b): Reducing vertex cover to set cover

```
def findVertexCover(G):
    V, E = G
    vertices = list(V)
    index = {v: i + 1 for i, v in enumerate(vertices)}
    U = set(range(len(E)))
    S = {i + 1: set() for i in range(len(vertices))}
    for edge, (u, v) in enumerate(E):
        S[index[u]].add(edge)
        S[index[v]].add(edge)
    C = findSetCover(U, S)
    return {vertices[i - 1] for i in C}
```

Each universe element identifies one graph edge. Subsets collect incident edges. The oracle returns selected subset indices. These indices map directly back to vertices. Every covered element therefore has a selected endpoint. Conversely, every vertex cover selects covering subsets. The two selections have equal sizes. Minimum set covers therefore yield minimum vertex covers. Decision instances keep the same budget k .

Assume E is an edge list. Construction takes expected $O(1 + |V| + |E|)$ time. The total incidence count is $2|E|$. The oracle takes $T_{SC}(U, S)$ time. Total time is $T_{SC}(U, S) + O(1 + |V| + |E|)$. This remains polynomial under the given assumption. Additional storage is $O(1 + |V| + |E|)$.